In [ ]:
```
1.String:Immutable,Ordered,Duplicates items are allowed,''," ",''''''," """"""
2.List:Mutable,Ordered,Duplicates items are allowed,[]
3.Tuple:Immutable,Ordered,Duplicates items are allowed,()
4.Set:Mutable,Unoredered,Duplicates items are not allowed,set(),{}
```

Set data Types: ¶

In [ ]:
```
Properties:
1)Muatble>> Changable(Add, delete ,update)
2)It is in Unordered(We can not use indexing and slicing )
3)Duplicate Items are nor allowed.
4)It is enclosed by curly brackets {}
5)initilize with set()
6)Set items are comma seperated.
```

In [2]:
```
set1={10,20,30,40,50}
set1
```

Out[2]:
```
{10, 20, 30, 40, 50}
```

In [3]:
```
set1[2]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 1
----> 1 set1[2]

TypeError: 'set' object is not subscriptable
```

In [4]:
```
list1=[10,20,30,40,50]
list1[2]
```

Out[4]:
```
30
```

In [5]:
```
set1={10,20,30,40,'python','machine','learning'}
set1
```

Out[5]:
```
{10, 20, 30, 40, 'learning', 'machine', 'python'}
```

In [6]:
```
set1={10,20,30,40,'python',20,'machine','learning'}
set1
```

Out[6]:
```
{10, 20, 30, 40, 'learning', 'machine', 'python'}
```

In [8]:
```
set1={10,20,30,40,'python',20,'PYTHON','machine','learning'}
set1
```

Out[8]:
```
{10, 20, 30, 40, 'PYTHON', 'learning', 'machine', 'python'}
```

In [9]:
```
a=[]
type(a)
```

Out[9]:
```
list
```

In [10]:
```
a=list()
type(a)
```

Out[10]:
```
list
```

In [11]:
```
b=()
type(b)
```

Out[11]:
```
tuple
```

In [12]:
```
c=""
type(c)
```

Out[12]:
```
str
```

In [13]:
```
c=str()
type(c)
```

Out[13]:
```
str
```

In [14]:
```
y=set()
type(y)
```

Out[14]:
```
set
```

In [16]:
```
s1={3,4,5,6,7.5,'python'}
s1
```

Out[16]:
```
{3, 4, 5, 6, 7.5, 'python'}
```

In [17]:
```
s1={3,4,5,6,7.5,'python',(10,20,30)}
s1
```

Out[17]:
```
{(10, 20, 30), 3, 4, 5, 6, 7.5, 'python'}
```

In [18]:
```
s1={3,4,5,6,7.5,'python',(10,20,30),[1,2,3]}
s1
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[18], line 1
----> 1 s1={3,4,5,6,7.5,'python',(10,20,30),[1,2,3]}
      2 s1

TypeError: unhashable type: 'list'
```

Type casting

In [ ]:
```
list>>tuple
list>>set

tuple>>list
tuple>>set

set>>list
set>>tuple
```

In [19]:
```
x=[2,3,4,5]
y=tuple(x)
y
```

Out[19]:
```
(2, 3, 4, 5)
```

In [20]:
```
x=[2,3,4,5]
y=set(x)
y
```

Out[20]:
```
{2, 3, 4, 5}
```

In [21]:
```
s1={2,3,4,5,6,7}
x=list(s1)
y=tuple(s1)
print(x)
print(y)
```

```
[2, 3, 4, 5, 6, 7]
(2, 3, 4, 5, 6, 7)
```

Access Set Items

In [22]:
```
s1={2,3,4,5,6,7}
for i in s1:
    print(f"i=={i}")
```

```
i==2
i==3
i==4
i==5
i==6
i==7
```

In [24]:
```
s1={10,20,'Machine Learning',"Pune",5.67,100}
print(s1)
for i in s1:
    print(i)
```

```
{'Machine Learning', 100, 5.67, 20, 10, 'Pune'}
Machine Learning
100
5.67
20
10
Pune
```

In [25]:
```
s1={10,20,'Machine Learning',"Pune",5.67,100}
for i,val in enumerate(s1):
    print(val)
```

```
Machine Learning
100
5.67
20
10
Pune
```

Set Functions

In [ ]:
```
1)add()
2)update()
3)union()

4)remove()
5)discard()
6)pop()
7)clear()

8)intersection()
9)intersection_update()

10)difference()
11)difference_update()

12)symmentric_difference
13)symmetric_difference_update()

14)issuperset()
15)issubset()
16)isdisjoint()
```

1.add()

In [ ]:
```
It is used to add only one item in set.
Set.add(Value)
```

In [26]:
```
s1={3,4,1,6,7,78,10,2,3,4}
s1.add(100)
s1
```

Out[26]: {1, 2, 3, 4, 6, 7, 10, 78, 100}

In [27]:
```
s1={2,4,5,6,7,8,2,3}
s1.add('python')
s1
```

Out[27]: {2, 3, 4, 5, 6, 7, 8, 'python'}

In [28]:
```
s1={2,4,5,6,7,8,2,3}
s1.add(6)
s1
```

Out[28]: {2, 3, 4, 5, 6, 7, 8}

In [29]:
```
s1={"pune","mumbai","delhi"}
s1.add("chennai")
s1
```

Out[29]: {'chennai', 'delhi', 'mumbai', 'pune'}

In [30]:
```
s1='python'
x=s1.replace('python','java')
x
```

Out[30]: 'java'

2)update()

In [ ]:
```
Set.update(iterable)
Iterable>>Set,list,tuple
```

In [31]:
```
x=[2,3,4,5]
y=[3,10,20,30]
x.extend(y)
x
```

Out[31]: [2, 3, 4, 5, 3, 10, 20, 30]

In [32]:
```
x={2,3,50,60,6,7}
y={10,20,30,40,50,30,40}
y.update(x)
x
```

Out[32]: {2, 3, 6, 7, 50, 60}

In [33]:
```
x={2,3,50,60,6,7}
y={10,20,30,40,50,30,40}
y.update(x)
y
```

Out[33]:
```
{2, 3, 6, 7, 10, 20, 30, 40, 50, 60}
```

In [34]:
```
x={2,3,50,60,6,7}
y={10,20,'30',40,50,30,40}
x.update(y)
x
```

Out[34]:
```
{10, 2, 20, 3, '30', 30, 40, 50, 6, 60, 7}
```

In [35]:
```
x={2,3,4,5,6}
y=[10,20,30,40,50]
x.update(y)
x
```

Out[35]:
```
{2, 3, 4, 5, 6, 10, 20, 30, 40, 50}
```

In [36]:
```
x={2,3,4,5,6}
y=[10,20,30,40,50]
y.update(x)
y
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[36], line 3
      1 x={2,3,4,5,6}
      2 y=[10,20,30,40,50]
----> 3 y.update(x)
      4 y

AttributeError: 'list' object has no attribute 'update'
```

In [37]:
```
x={2,3,4,5,6}
y=[10,20,30,40,50]
z=('python','python','data science')
x.update(z)
x
```

Out[37]:
```
{2, 3, 4, 5, 6, 'data science', 'python'}
```

In [38]:
```
x={2,3,4,5,6}
y=[10,20,30,40,50]
z=('python','python','data science')
z.update(x)
z
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[38], line 4
      2 y=[10,20,30,40,50]
      3 z=('python','python','data science')
----> 4 z.update(x)
      5 z

AttributeError: 'tuple' object has no attribute 'update'
```

3)union()

In [ ]:
```
Return a set containing the union of sets
New_set=Set.union(iterable)
ietrable>>set,tuple,list
```

In [43]:
```
s1={3,4,5,6,7,8,5,3}
s2={10,20,30,40,50,20,30}
s1.update(s2)
print("set 1:",s1)
print("Set 2:",s2)
```

```
set 1: {3, 4, 5, 6, 7, 8, 40, 10, 50, 20, 30}
Set 2: {50, 20, 40, 10, 30}
```

In [41]:
```
s1={3,4,5,6,7,8,5,3}
s2={10,20,30,40,50,20,30}
new_set=s1.union(s2)
print("Set 1:",s1)
print("Set 2:",s2)
print("New Set:",new_set)
```

```
Set 1: {3, 4, 5, 6, 7, 8}
Set 2: {50, 20, 40, 10, 30}
New Set: {3, 4, 5, 6, 7, 8, 40, 10, 50, 20, 30}
```

Delete Items from Set

In [ ]:
```
1)remove()
2)discard()
3)pop()
4)clear()
```

1)remove()

In [ ]:
```
Set.remove(value)
```

In [1]:
```
s1={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
print(s1)
s1.remove('Mumbai')
s1
```

```
{'Delhi', 'Pune', 'Hydrabad', 'Chennai', 'Mumbai'}
```

Out[1]:
```
{'Chennai', 'Delhi', 'Hydrabad', 'Pune'}
```

In [2]:
```
s1={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
s1.remove('Banglore')
s1
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[2], line 2
      1 s1={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
----> 2 s1.remove('Banglore')
      3 s1

KeyError: 'Banglore'
```

In [3]:
```
s1={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
city='Mumbai'
if city in s1:
    s1.remove(city)
    print('Item deleted')
```

```
Item deleted
```

In [5]:
```
s1={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
city='Banglore'
if city in s1:
    s1.remove(city)
    print('Item deleted')
else:
    print("Element Not found")
```

```
Element Not found
```

2)discard()

In [ ]:
```
Set.discard(value)
Returns original set if item is not present in given set.
```

In [16]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai','Pune'}
s2.remove('Pune')
s2
```

Out[16]:
```
{'Chennai', 'Delhi', 'Hydrabad', 'Mumbai'}
```

In [17]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai','Pune'}
s2.remove('PUNE')
s2
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[17], line 2
      1 s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai','Pune'}
----> 2 s2.remove('PUNE')
      3 s2

KeyError: 'PUNE'
```

In [7]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
s2.discard('NAgar')
s2
```

Out[7]:
```
{'Chennai', 'Delhi', 'Hydrabad', 'Mumbai', 'Pune'}
```

3)pop()

In [ ]:
```
Set.pop()
It will delete random item from set.
```

In [15]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai','Pune'}
s2.pop()
s2
```

Out[15]:
```
{'Chennai', 'Hydrabad', 'Mumbai', 'Pune'}
```

In [9]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
s2.pop()
s2
```

Out[9]:
```
{'Chennai', 'Hydrabad', 'Mumbai', 'Pune'}
```

In [11]:
```
s3={3,4,5,6,7,8}
s3.pop()
s3
```

Out[11]:
```
{4, 5, 6, 7, 8}
```

4)clear()

In [ ]:
```
Return>>set()>>Empty Set
It will dlete all items from set.
```

In [12]:
```
s2={'Pune','Mumbai','Delhi','Hydrabad','Chennai'}
s2.clear()
s2
```

Out[12]:
```
set()
```

In [14]:
```
s3={3,4,5,6,7,8,'python',5}
s3.clear()
s3
```

Out[14]:
```
set()
```

1.Intersection

In [ ]:
```
Set.intersection(iterable)
iterable>>list,set,tuple
```

In [22]:
```
list1=[2,3,7,8,9,10]
list2=[4,5,6,7,8,1,2,3,5,9,11,12]
for i in list1:
    if i in list2:
        print(f"i=={i}")
```

```
i==2
i==3
i==7
i==8
i==9
```

In [19]:
```
list1=[2,3,7,8,9,10]
list2=[4,5,6,7,8]
result=[]
for i in list1:
    if i in list2:
        result.append(i)
print(result)
```

```
[7, 8]
```

In [20]:
```
list1=[2,3,7,8,9,10]
list2=[4,5,6,7,8]
result=[i for i in list1 if i in list2]
result
```

Out[20]:
```
[7, 8]
```

In [21]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
result=set1.intersection(set2)

print(set1)
print(set2)
print(result)
```

```
{2, 3, 7, 8, 9, 10}
{4, 5, 6, 7, 8}
{8, 7}
```

In [23]:
```
list1=[2,3,7,8,9,10]
list2=[4,5,6,7,8]
x=set(list1).intersection(list2)
x
```

Out[23]: {7, 8}

In [26]:
```
list1=[2,3,7,8,9,10]
list2=[4,5,6,7,8]
list3=[8,7,10,20,30,40]
s2=set(list2)
s2.intersection(list1,list3)
```

Out[26]: {7, 8}

2.Intersection_update

In [27]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set1.intersection_update(set2)

print(set1)
print(set2)
```

```
{8, 7}
{4, 5, 6, 7, 8}
```

In [28]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set2.intersection_update(set1)

print(set1)
print(set2)
```

```
{2, 3, 7, 8, 9, 10}
{8, 7}
```

In [30]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
x=set2.intersection_update(set1)

print(set1)
print(set2)
print(x)   #it will not not return new set
```

```
{2, 3, 7, 8, 9, 10}
{8, 7}
None
```

3.difference

In [31]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set3=set1.difference(set2)
set3
```

Out[31]: {2, 3, 9, 10}

In [32]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set3=set2.difference(set1)
set3
```

Out[32]: {4, 5, 6}

4.difference_update

In [33]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set2.difference_update(set1)

print(set1)
print(set2)
```

```
{2, 3, 7, 8, 9, 10}
{4, 5, 6}
```

In [34]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set1.difference_update(set2)

print(set1)
print(set2)
```

```
{2, 3, 9, 10}
{4, 5, 6, 7, 8}
```

5.Symmetric_difference

In [ ]:
```
Set1.symmetric_difference(set2)

Unique items from two sets
```

In [36]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set1.symmetric_difference(set2)
```

Out[36]:
```
{2, 3, 4, 5, 6, 9, 10}
```

In [40]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set2.symmetric_difference(set1)

print(set1)
print(set2)
```

```
{2, 3, 7, 8, 9, 10}
{4, 5, 6, 7, 8}
```

6.Symmetric_difference_update

In [38]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set2.symmetric_difference_update(set1)

print(set1)
print(set2)
```

```
{2, 3, 7, 8, 9, 10}
{2, 3, 4, 5, 6, 9, 10}
```

In [39]:
```
set1={2,3,7,8,9,10}
set2={4,5,6,7,8}
set1.symmetric_difference_update(set2)

print(set1)
print(set2)
```

```
{2, 3, 4, 5, 6, 9, 10}
{4, 5, 6, 7, 8}
```

6.issuperset()

In [ ]:
```
set1.issuperset(set2)
True or False
```

In [41]:
```
set1={2,3,7,8,9,10}
set2={3,9,7}
set1.issuperset(set2)
```

Out[41]: True

In [42]:
```
set1={2,3,7,8,9,10}
set2={3,9,6}
set1.issuperset(set2)
```

Out[42]: False

7.issubset()

In [43]:
```
set1={2,3,7,8,9,10}
set2={3,9,7}
set2.issubset(set1)
```

Out[43]: True

In [44]:
```
set1={2,3,7,8,9,10}
set2={3,9,7}
set1.issubset(set2)
```

Out[44]: False

8.isdisjoint()

In [45]:
```
set1={2,3,7,8,9,10}
set2={100,200,300,400}
set1.isdisjoint(set2)
```

Out[45]: True

In [46]:
```
set1={2,3,7,8,9,10}
set2={3,9,7,20,40,50}
set1.isdisjoint(set2)
```

Out[46]: False

In [ ]:
```

```